The normal distribution

Lionel jr desroses

In this lab, you'll investigate the probability distribution that is most central to statistics: the normal distribution. If you are confident that your data are nearly normal, that opens the door to many powerful statistical methods. Here we'll use the graphical tools of R to assess the normality of our data and also learn how to generate random numbers from a normal distribution.

Getting Started

Load packages

In this lab, we will explore and visualize the data using the **tidyverse** suite of packages as well as the **openintro** package.

Let's load the packages.

```
library(tidyverse)
library(openintro)
library(ggplot2)
```

The data

This week you'll be working with fast food data. This data set contains data on 515 menu items from some of the most popular fast food restaurants worldwide. Let's take a quick peek at the first few rows of the data.

Either you can use `glimpse` like before, or `head` to do this.

```
library(tidyverse)
library(openintro)
data("fastfood", package='openintro')
head(fastfood)
```

```
## # A tibble: 6 x 17
##   restaurant item       calories cal_fat total_fat sat_fat trans_fat cholesterol
##   <chr>      <chr>         <dbl>   <dbl>     <dbl>   <dbl>     <dbl>       <dbl>
## 1 Mcdonalds  Artisan G~      380      60         7       2       0            95
## 2 Mcdonalds  Single Ba~      840     410        45      17       1.5         130
## 3 Mcdonalds  Double Ba~     1130     600        67      27       3           220
## 4 Mcdonalds  Grilled B~      750     280        31      10       0.5         155
## 5 Mcdonalds  Crispy Ba~      920     410        45      12       0.5         120
## 6 Mcdonalds  Big Mac          540     250        28      10       1            80
## # i 9 more variables: sodium <dbl>, total_carb <dbl>, fiber <dbl>, sugar <dbl>,
## #   protein <dbl>, vit_a <dbl>, vit_c <dbl>, calcium <dbl>, salad <chr>
```

You'll see that for every observation there are 17 measurements, many of which are nutritional facts.

You'll be focusing on just three columns to get started: restaurant, calories, calories from fat.

Let's first focus on just products from McDonalds and Dairy Queen.

```
mcdonalds <- fastfood %>%
  filter(restaurant == "Mcdonalds")
dairy_queen <- fastfood %>%
  filter(restaurant == "Dairy Queen")
```

1. Make a plot (or plots) to visualize the distributions of the amount of calories from fat of the options from these two restaurants. How do their centers, shapes, and spreads compare?

Insert your answer here

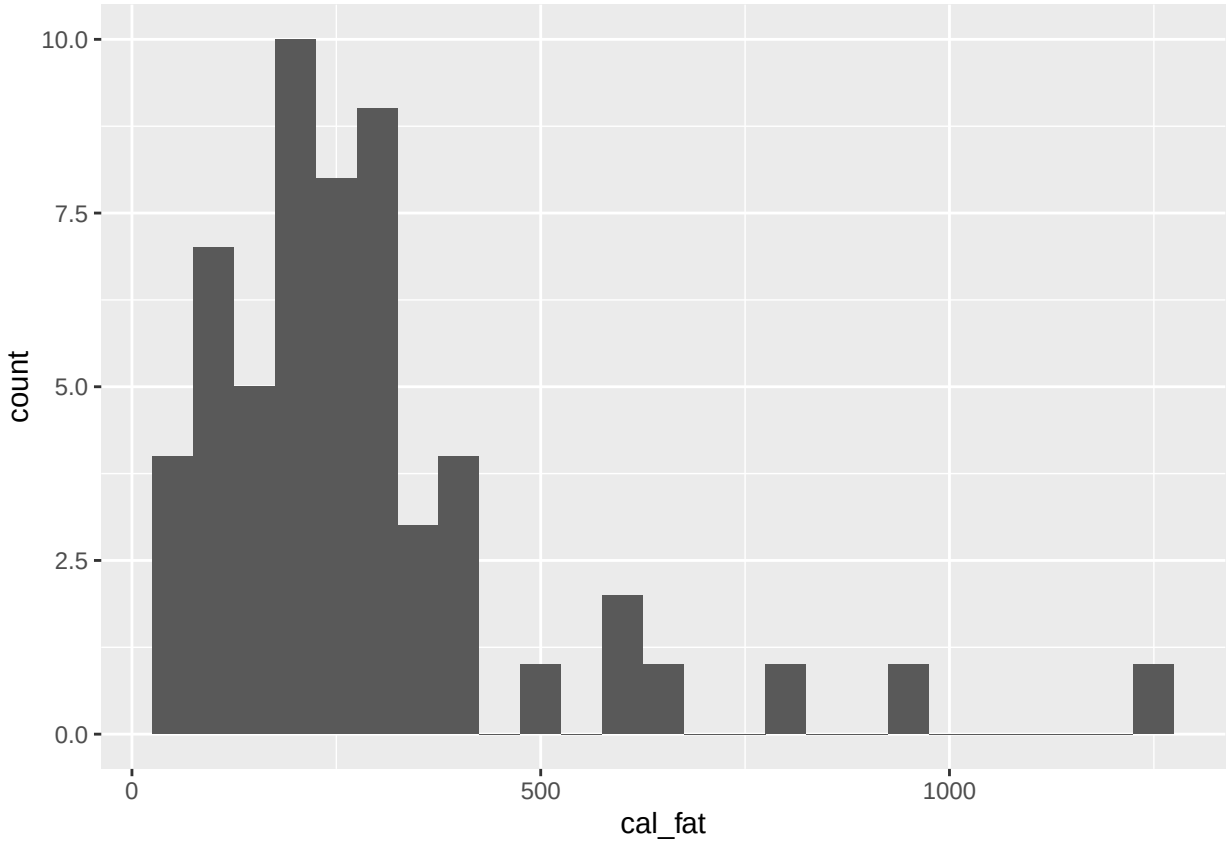
```
dairy_queen$cal_fat
```

```
##  [1] 660 460 330 270 310 160 310 480 590 240 220 220 180 160 180  80  80 410 670
## [20] 440 140 200 160 310 240 130 430 310 430 180 180 220 280 120 270 190 170  20
## [39] 140 130   0 240
```

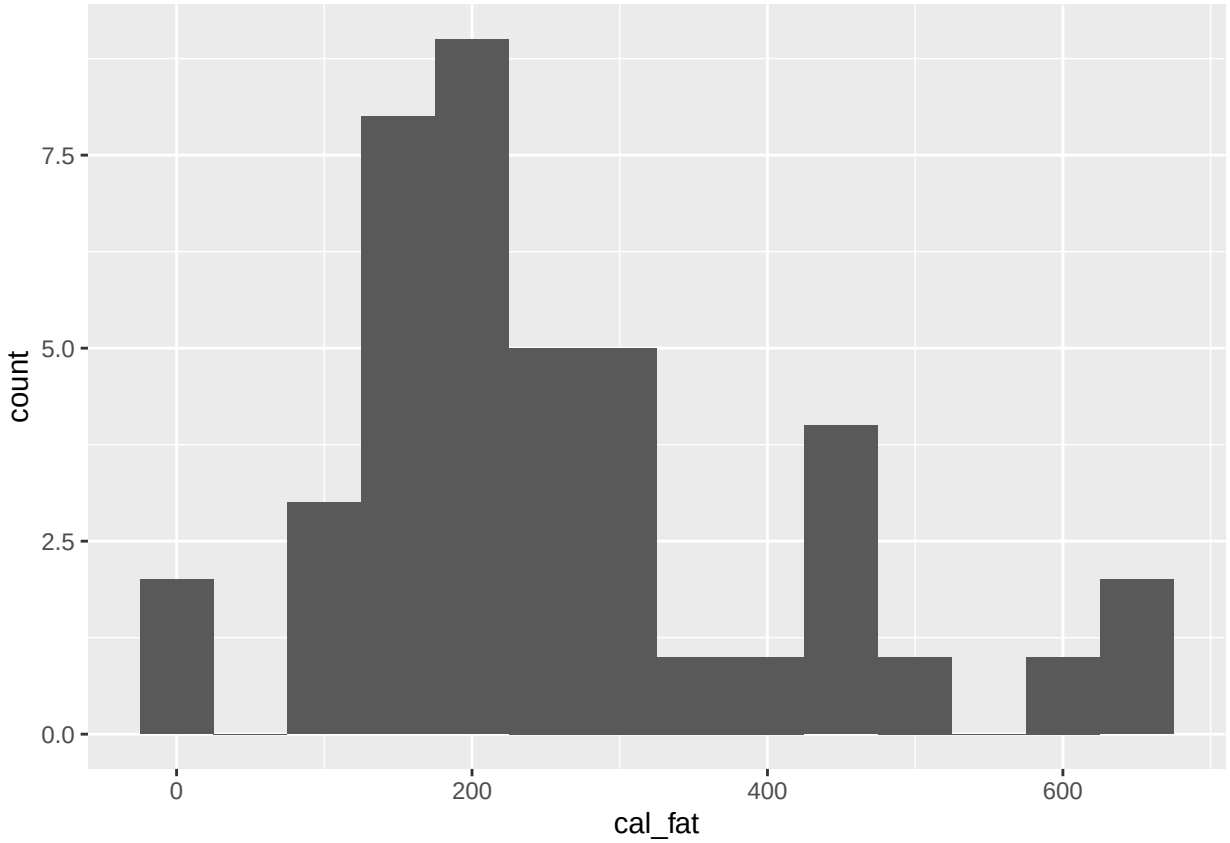
```
mcdonalds$cal_fat
```

```
##  [1]   60  410  600  280  410  250  100  210  190  400  170  300  180  300   70
## [16]   50  330  190  310  130  160  200  300  160  280  200  200  240  320  180
## [31]  300  340  200  320  190  250  390  630  790 1270  100  140  240  480  960
## [46]  240  360  600   70   80  250  110  120  250   90  100  230
```

```
ggplot(mcdonalds, aes(x = cal_fat)) +
 geom_histogram(binwidth = 50)
```




```
ggplot(dairy_queen, aes(x = cal_fat)) +
 geom_histogram(binwidth = 50)
```



The distributions of calories from fat for both McDonald's and Dairy Queen are unimodal and somewhat right-skewed. Dairy Queen appears to have a higher center, meaning its menu items generally contain more calories from fat.

The normal distribution

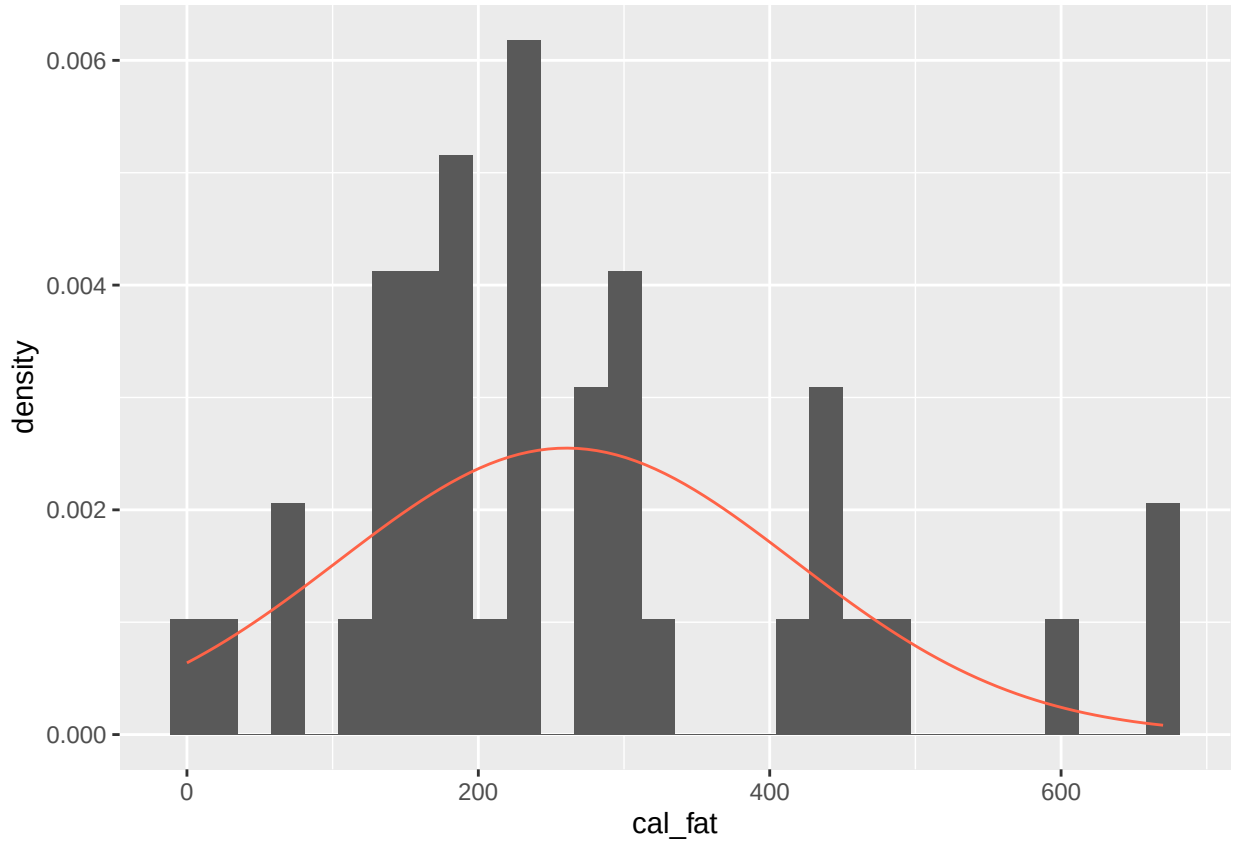
In your description of the distributions, did you use words like *bell-shaped*or *normal*? It's tempting to say so when faced with a unimodal symmetric distribution.

To see how accurate that description is, you can plot a normal distribution curve on top of a histogram to see how closely the data follow a normal distribution. This normal curve should have the same mean and standard deviation as the data. You'll be focusing on calories from fat from Dairy Queen products, so let's store them as a separate object and then calculate some statistics that will be referenced later.

```
dqmean <- mean(dairy_queen$cal_fat)
dqsd   <- sd(dairy_queen$cal_fat)
```

Next, you make a density histogram to use as the backdrop and use the `lines` function to overlay a normal probability curve. The difference between a frequency histogram and a density histogram is that while in a frequency histogram the *heights* of the bars add up to the total number of observations, in a density histogram the *areas* of the bars add up to 1. The area of each bar can be calculated as simply the height *times* the width of the bar. Using a density histogram allows us to properly overlay a normal distribution curve over the histogram since the curve is a normal probability density function that also has area under the curve of 1. Frequency and density histograms both display the same exact shape; they only differ in their y-axis. You can verify this by comparing the frequency histogram you constructed earlier and the density histogram created by the commands below.

```
ggplot(data = dairy_queen, aes(x = cal_fat)) +
        geom_blank() +
        geom_histogram(aes(y = ..density..)) +
        stat_function(fun = dnorm, args = c(mean = dqmean, sd = dqsd), col = "tomato")
```



After initializing a blank plot with `geom_blank()`, the `ggplot2` package (within the `tidyverse`) allows us to add additional layers. The first layer is a density histogram. The second layer is a statistical function – the density of the normal curve, `dnorm`. We specify that we want the curve to have the same mean and standard deviation as the column of fat calories. The argument `col` simply sets the color for the line to be drawn. If we left it out, the line would be drawn in black.

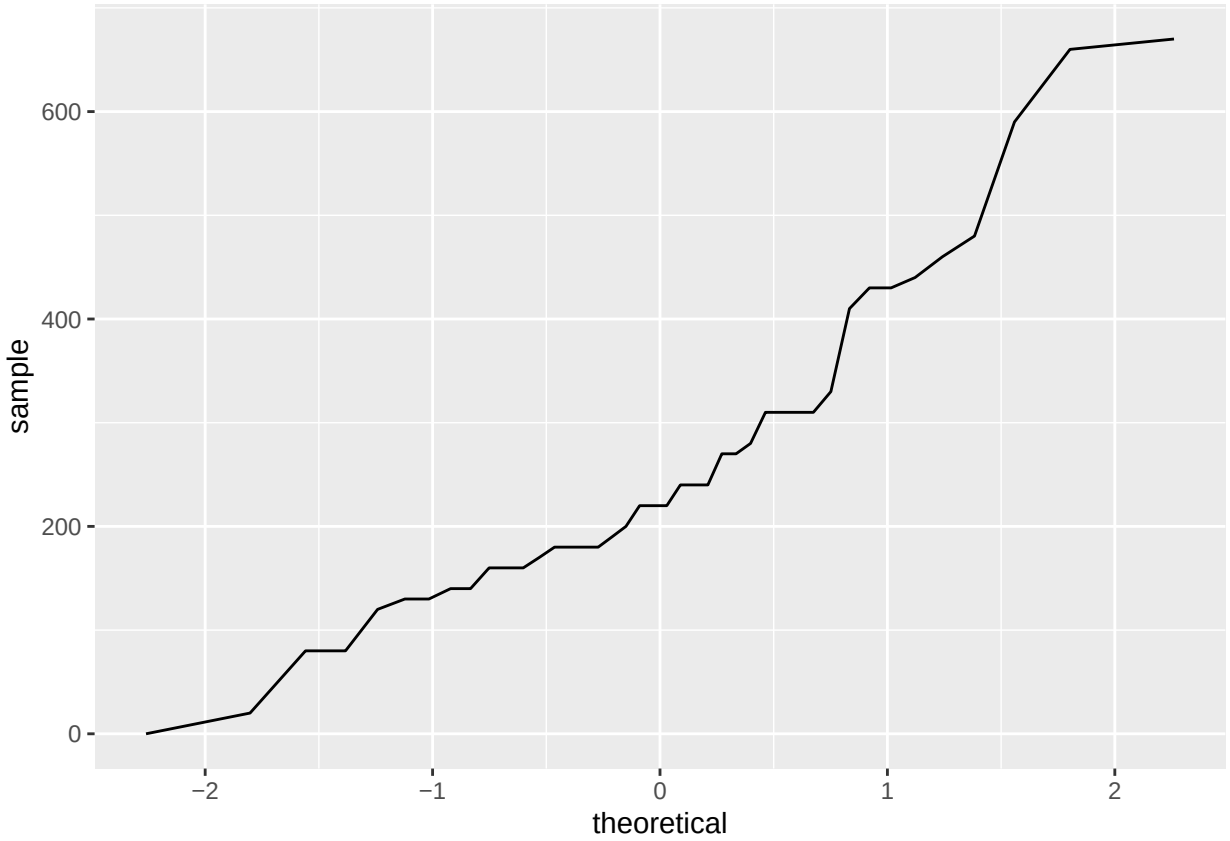
2. Based on the this plot, does it appear that the data follow a nearly normal distribution?

Insert your answer here The distribution appears roughly bell-shaped in the center, but it does not perfectly follow a normal distribution. The histogram shows some right skewness and several observations in the upper tail that fall above what would be expected under a normal model. Therefore, the data are only approximately normal and not perfectly normal.

Evaluating the normal distribution

Eyeballing the shape of the histogram is one way to determine if the data appear to be nearly normally distributed, but it can be frustrating to decide just how close the histogram is to the curve. An alternative approach involves constructing a normal probability plot, also called a normal Q-Q plot for "quantile-quantile".

```
ggplot(data = dairy_queen, aes(sample = cal_fat)) +
  geom_line(stat = "qq")
```



This time, you can use the `geom_line()` layer, while specifying that you will be creating a Q-Q plot with the `stat` argument. It's important to note that here, instead of using `x` instead `aes()`, you need to use `sample`.

The x-axis values correspond to the quantiles of a theoretically normal curve with mean 0 and standard deviation 1 (i.e., the standard normal distribution). The y-axis values correspond to the quantiles of the original unstandardized sample data. However, even if we were to standardize the sample data values, the Q-Q plot would look identical. A data set that is nearly normal will result in a probability plot where the points closely follow a diagonal line. Any deviations from normality leads to deviations of these points from that line.

The plot for Dairy Queen's calories from fat shows points that tend to follow the line but with some errant points towards the upper tail. You're left with the same problem that we encountered with the histogram above: how close is close enough?

A useful way to address this question is to rephrase it as: what do probability plots look like for data that I *know* came from a normal distribution? We can answer this by simulating data from a normal distribution using `rnorm`.

```
sim_norm <- rnorm(n = nrow(dairy_queen), mean = dqmean, sd = dqsd)
```

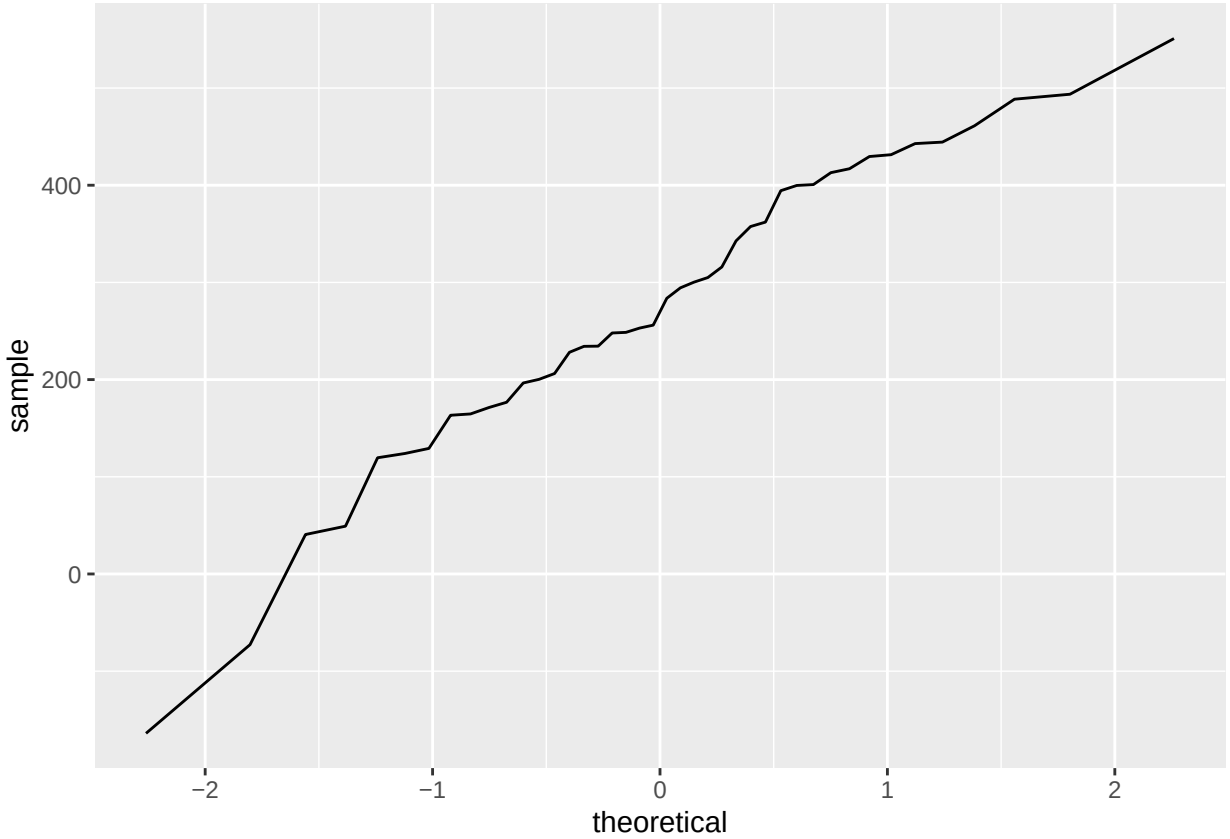
The first argument indicates how many numbers you'd like to generate, which we specify to be the same number of menu items in the `dairy_queen` data set using the `nrow()` function. The last two arguments determine the mean and standard deviation of the normal distribution from which the simulated sample will

be generated. You can take a look at the shape of our simulated data set, `sim_norm`, as well as its normal probability plot.

3. Make a normal probability plot of `sim_norm`. Do all of the points fall on the line? How does this plot compare to the probability plot for the real data? (Since `sim_norm` is not a data frame, it can be put directly into the `sample` argument and the `data` argument can be dropped.)

Insert your answer here

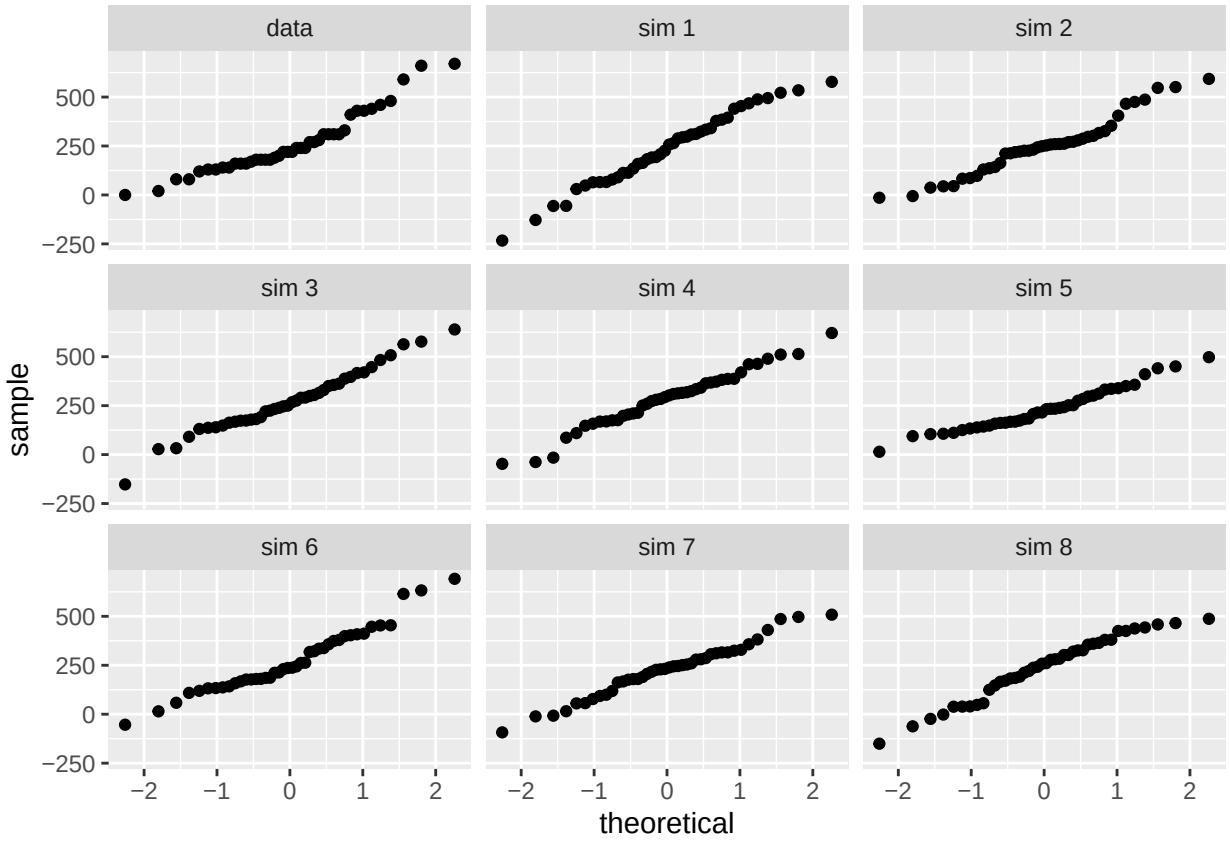
```
ggplot(data = NULL, aes(sample = sim_norm)) +
 geom_line(stat = "qq")
```



Answer- No, not all of the points fall exactly on the line. Even data generated from a normal distribution will show random variation. However, the points generally follow the diagonal line much more closely than the Dairy Queen data.

Even better than comparing the original plot to a single plot generated from a normal distribution is to compare it to many more plots using the following function. It shows the Q-Q plot corresponding to the original data in the top left corner, and the Q-Q plots of 8 different simulated normal data. It may be helpful to click the zoom button in the plot window.

```
qqnormsim(sample = cal_fat, data = dairy_queen)
```



4. Does the normal probability plot for the calories from fat look similar to the plots created for the simulated data? That is, do the plots provide evidence that the calories are nearly normal?

Insert your answer here Answer- The Dairy Queen Q-Q plot is similar to the simulated normal plots in the middle portion of the distribution. However, there are noticeable deviations in the tails, particularly in the upper tail.

5. Using the same technique, determine whether or not the calories from McDonald's menu appear to come from a normal distribution.

Insert your answer here

Answer -The McDonald's calories-from-fat distribution does not appear to come from a perfectly normal distribution. The Q-Q plot shows departures from the diagonal line, especially in the tails. While the center of the distribution aligns reasonably well with the line, the skewness and tail behavior suggest that the data are only approximately normal

Normal probabilities

Okay, so now you have a slew of tools to judge whether or not a variable is normally distributed. Why should you care?

It turns out that statisticians know a lot about the normal distribution. Once you decide that a random variable is approximately normal, you can answer all sorts of questions about that variable related to probability. Take, for example, the question of, "What is the probability that a randomly chosen Dairy Queen product has more than 600 calories from fat?"

If we assume that the calories from fat from Dairy Queen's menu are normally distributed (a very close approximation is also okay), we can find this probability by calculating a Z score and consulting a Z table (also called a normal probability table). In R, this is done in one step with the function `pnorm()`.

```
1 - pnorm(q = 600, mean = dqmean, sd = dqsd)
```

```
## [1] 0.01501523
```

Note that the function `pnorm()` gives the area under the normal curve below a given value, `q`, with a given mean and standard deviation. Since we're interested in the probability that a Dairy Queen item has more than 600 calories from fat, we have to take one minus that probability.

Assuming a normal distribution has allowed us to calculate a theoretical probability. If we want to calculate the probability empirically, we simply need to determine how many observations fall above 600 then divide this number by the total sample size.

```
dairy_queen %>%
  filter(cal_fat > 600) %>%
  summarise(percent = n() / nrow(dairy_queen))
```

```
## # A tibble: 1 x 1
##   percent
##     <dbl>
## 1  0.0476
```

Although the probabilities are not exactly the same, they are reasonably close. The closer that your distribution is to being normal, the more accurate the theoretical probabilities will be.

6. Write out two probability questions that you would like to answer about any of the restaurants in this dataset. Calculate those probabilities using both the theoretical normal distribution as well as the empirical distribution (four probabilities in all). Which one had a closer agreement between the two methods?

Insert your answer here q1- What is the probability that a Dairy Queen menu item has more than 500 calories from fat?

```
pnorm(500, mean = dqmean, sd = dqsd)
```

```
## [1] 0.937072
```

```
dairy_queen %>%
 summarise(prob = mean(cal_fat > 500))
```

```
## # A tibble: 1 x 1
##     prob
##    <dbl>
## 1 0.0714
```

q2-What is the probability that a McDonald's menu item has fewer than 200 calories from fat?

```
pnorm(200,
   mean = mean(mcdonalds$cal_fat),
   sd = sd(mcdonalds$cal_fat))
```

```
## [1] 0.349167
```

```
mcdonalds %>%
 summarise(prob = mean(cal_fat < 200))
```

```
## # A tibble: 1 x 1
##    prob
##   <dbl>
## 1 0.368
```

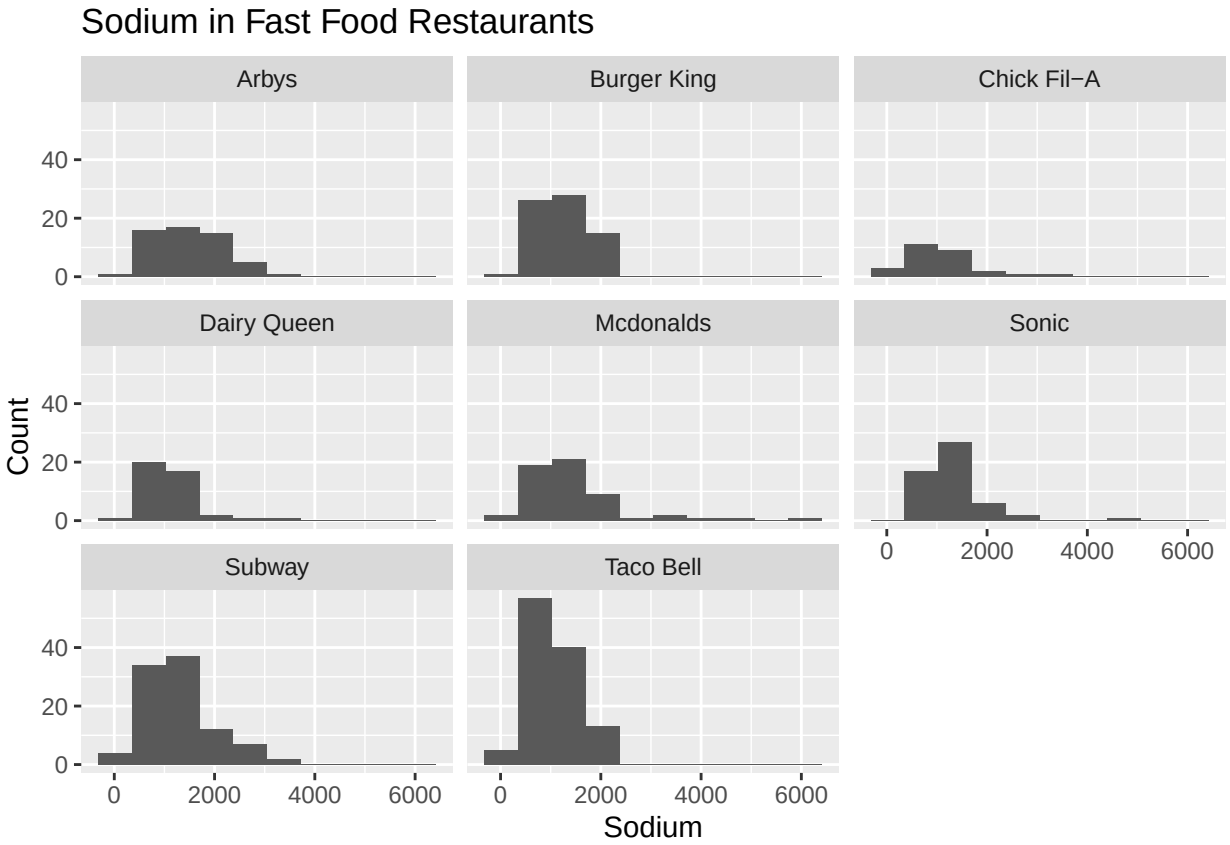
Answer–The theoretical probabilities were reasonably close to the empirical probabilities, although they were not identical. The question involving the value closer to the center of the distribution showed better agreement because the normal approximation works best near the center and less well in the tails. * * *

More Practice

7. Now let's consider some of the other variables in the dataset. Out of all the different restaurants, which ones' distribution is the closest to normal for sodium?

Insert your answer here

```
fastfood %>%
  group_by(restaurant) %>%
  ggplot() +
  geom_histogram(aes(x = sodium), bins = 10) +
  ggtitle("Sodium in Fast Food Restaurants") +
  xlab("Sodium") +
  ylab("Count") +
  facet_wrap(. ~restaurant)
```



8. Note that some of the normal probability plots for sodium distributions seem to have a stepwise pattern. why do you think this might be the case?

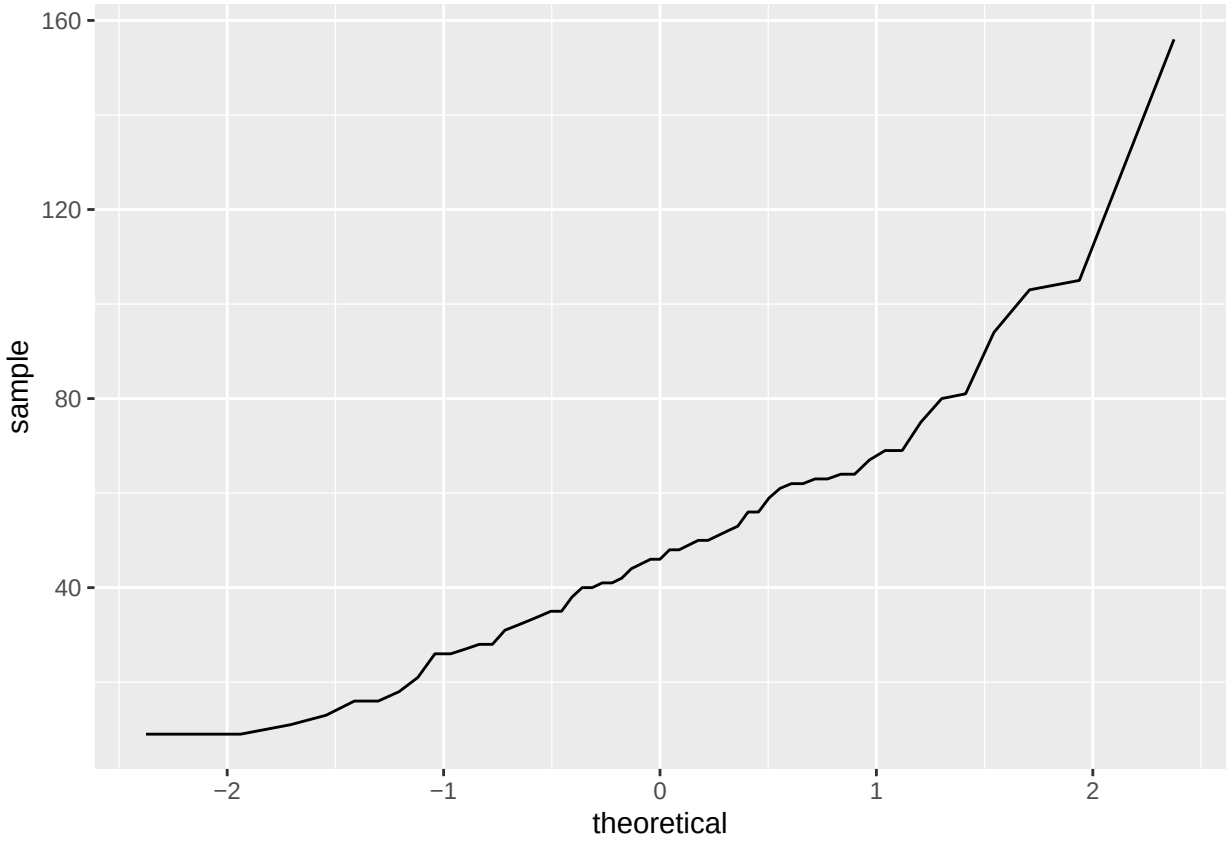
Insert your answer here

Answer -The stepwise pattern occurs because many sodium values are repeated. Sodium content is often reported in rounded numbers such as 500 mg, 750 mg, or 1000 mg. Since many menu items share identical sodium values, the quantiles are repeated, producing horizontal steps in the Q-Q plot rather than a smooth pattern.

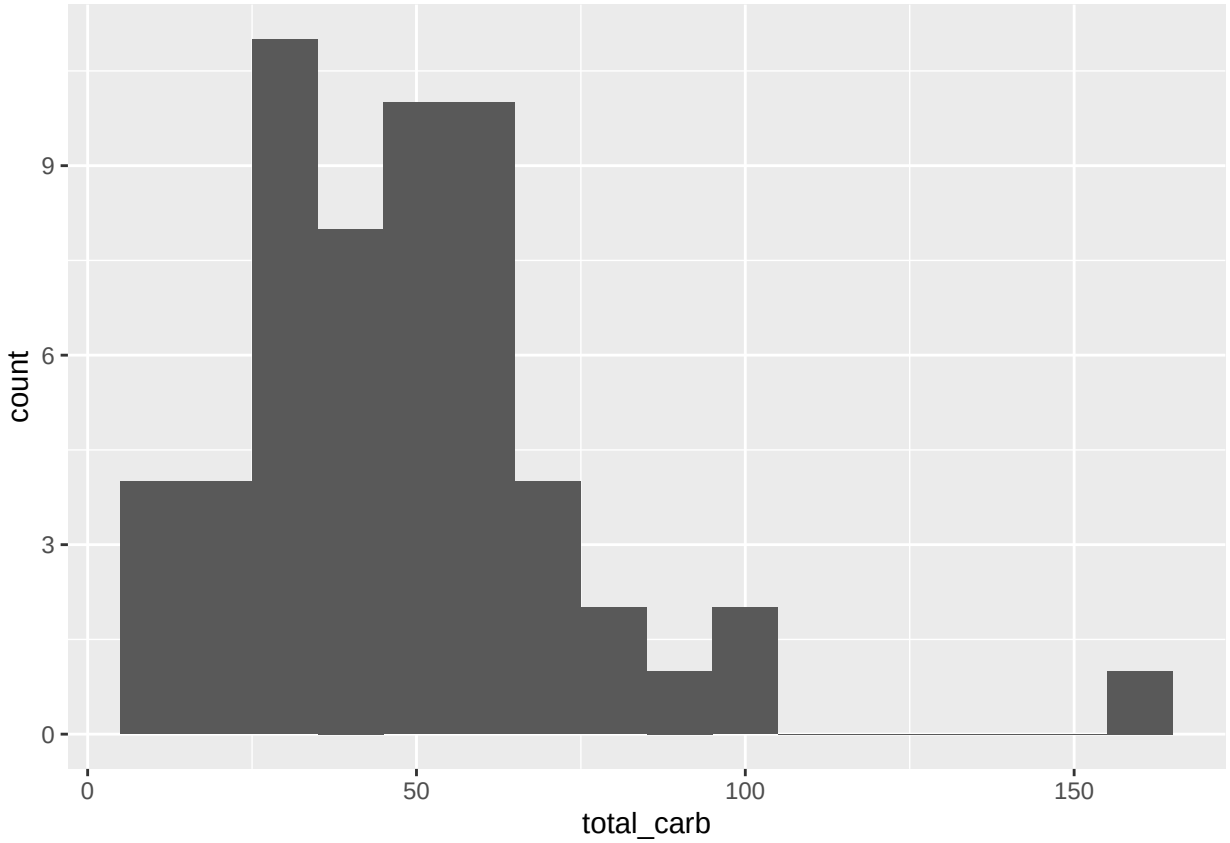
9. As you can see, normal probability plots can be used both to assess normality and visualize skewness. Make a normal probability plot for the total carbohydrates from a restaurant of your choice. Based on this normal probability plot, is this variable left skewed, symmetric, or right skewed? Use a histogram to confirm your findings.

Insert your answer here

```
ggplot(mcdonalds, aes(sample = total_carb)) +
 geom_line(stat = "qq")
```




```
ggplot(mcdonalds, aes(x = total_carb)) +
 geom_histogram(binwidth = 10)
```



Answer-The total carbohydrate distribution appears to be right-skewed. In the Q-Q plot, the upper tail bends upward away from the reference line, which is characteristic of right-skewed data. The histogram confirms this pattern by showing a longer tail extending toward higher carbohydrate values. Therefore, the variable is not symmetric and exhibits positive skewness. * * *